Regresión del DirichletCoupling heredado en HyperFEM.jl

Informe técnico — HyperFEMKernels.jl, ejemplo examples/hmre_vacuum_2d · 21 de octubre de 2026 · Jesús Martínez-Frutos (MultiSimO Lab, UPCT), con asistencia de Claude

Resumen

El refactor de las condiciones de contorno de HyperFEM (commits 3e005d3 → e161faa, 9–13 de octubre de 2026, ya en `main`) cambia la forma en que se aplica un `DirichletCoupling` rellenado con `InterpolableBC!`. El valor acoplado pasa de escribirse por máscara en los grados de libertad de la etiqueta a entregarse a Gridap como una función del espacio, `x → evaluate(uh, x)`. Cuando uh es un campo definido únicamente sobre la interfaz (el caso de uso real de `hMRE_Vacuum_2D.jl`), Gridap lo evalúa en nodos de todo el dominio y la búsqueda de puntos falla. El script original de Adv_Mater_Ultra_Soft_Magnet ya no se ejecuta con el HyperFEM actual. Además, la evaluación punto a punto reconstruye la caché de búsqueda en cada llamada (0,78 s y 1,4 GB por actualización de los valores Dirichlet del vacío).

1. El acoplamiento en el ejemplo

En `hMRE_Vacuum_2D.jl` el desplazamiento del sólido se impone al vacío a través de la interfaz:

```
dir_u_values_air = [[0.0, 0.0], [0.0, 0.0], DirichletCoupling]
Du_vacuum = DirichletBC(["Fixed_Air", "Fixed_Air_x", "Interface"], dir_u_values_air, [evolu, evolu, evolu])
...
Uair_int   = FESpace(Γair_int, reffeu_vacuum)          # espacio FE SOLO sobre la interfaz
Usolid_int = FESpace(Γsolid_int, reffeu)
uhair_int_(Λ) = interpolate_everywhere!(uhsolid_int_(Λ), get_free_dof_values(uhair_int),
                                        uhair_int.dirichlet_values, Uair_int)
InterpolableBC!(Uu_vacuum⁺, Du_vacuum, "Interface", uhair_int_)
```

`uhair_int_(Λ)` devuelve una `FEFunction` de `Uair_int`, un espacio construido sobre la `BoundaryTriangulation` de la interfaz. Ese campo solo existe en las caras de la interfaz; fuera de ellas no hay celda activa en la que evaluarlo.

2. Comportamiento anterior (hasta fa414c6, 8 de octubre)

`InterpolableBC` guardaba, en su constructor, las coordenadas de los nodos Dirichlet de la etiqueta `"Interface"` (`coords`) y una máscara sobre `dirichlet_values`. En cada `TrialFESpace!` se evaluaba el campo únicamente en esas coordenadas y se copiaban los valores en las posiciones de la máscara, después de la interpolación normal de Gridap para el resto de etiquetas:

```
# FESpaces.jl (fa414c6)
function TrialFESpace!(space, bc::DirichletBC, Λ)
  ...
  TrialFESpace!(space, values)                 # Gridap: etiquetas normales; el coupling recibe (x) -> x
  for i in eachindex(bc.tags)
    if bc.caches[i] isa InterpolableBC
      bc.caches[i](Λ)                          # evaluate(Interpolable(Λ), coords)  — solo nodos de la etiqueta
      space.dirichlet_values[bc.caches[i].caches[2]] = bc.caches[i].caches[1]
    end
  end
end
```

3. Comportamiento actual (e161faa)

El marcador `DirichletCoupling` se traduce a `Parametric(LegacyCoupling(nothing))`, e `InterpolableBC!` rellena en sitio su campo `Interpolable`. Al evaluar la prescripción, la etiqueta recibe una función del espacio:

```
# BoundaryConditions.jl (e161faa)
(c::LegacyCoupling)(Λ) = isnothing(c.Interpolable) ? identity : _evaluator(c.Interpolable(Λ))
_evaluator(f) = x -> evaluate(f, x)
...
get_space_functions(bc::BoundaryCondition, Λ) = bc.value(Λ)
TrialFESpace!(space, bc::BoundaryConditions, Λ) = TrialFESpace!(space, get_space_functions(bc, Λ))
```

Gridap interpola cada función de Dirichlet en todas las celdas del espacio y después se queda con los dofs de su etiqueta. Por tanto `x → evaluate(uh, x)` se llama también en nodos del interior del vacío, donde uh no está definida:

```
ERROR: LoadError: AssertionError: Point (0.069, -0.1) is not inside any active cell
  _point_to_cell!  Gridap/src/CellData/Interpolation.jl:154
  evaluate         Gridap/src/Arrays/Maps.jl:87
  #_evaluator##0   HyperFEM.jl/src/ComputationalModels/BoundaryConditions.jl:364
  TrialFESpace!    HyperFEM.jl/src/ComputationalModels/FESpaces.jl:9
  StaticNonlinearModel  HyperFEM.jl/src/ComputationalModels/Drivers.jl:114
```

La clase `InterpolableBC` sigue existiendo con sus `coords` y `caches`, pero su resultado ya no se utiliza en `TrialFESpace!`.

4. Coste, aun cuando no falla

Si se evita el fallo devolviendo un valor fuera de la interfaz, queda el segundo problema: `evaluate(uh, x)` punto a punto construye en cada llamada la caché de búsqueda de Gridap (KD-tree sobre la triangulación). Medido en el Mac (M5 Max, 18 hilos) con `dev/prof_hmre_vacuum_bc.jl`:

Espacio	dofs libres	dofs Dirichlet	TrialFESpace! (s)	Memoria
magnético (φ)	8 563	72	0,000	0,2 MB
sólido (u)	1 232	16	0,000	0,04 MB
vacío (u), evaluación sin caché	15 580	852	0,780	1 366 MB
vacío (u), con caché (apaño)	15 580	852	0,006	19 MB

En el bucle escalonado el espacio del vacío se actualiza varias veces por paso (TrialFESpace! en el driver, en `dirichlet_preconditioning!` y en `StaggeredModel`): sin caché, esa actualización dominaba el 93 % del tiempo de la simulación (perfil con `dev/prof_hmre_vacuum_solve.jl`).

5. Por qué el test de HyperFEM no lo detecta

`test/ComputationalModelsTests/BoundaryConditionsTests.jl` prueba el acoplamiento heredado con un campo uhA interpolado en una malla cartesiana que cubre todo el dominio. La evaluación fuera de la etiqueta devuelve un valor (que luego se descarta) y el test pasa. El caso real —campo definido sobre un `FESpace(BoundaryTriangulation(...))`— no está cubierto.

6. Apaño local en el ejemplo de HyperFEMKernels

Para poder ejecutar el ejemplo sin tocar HyperFEM, `examples/hmre_vacuum_2d/hmre_vacuum_2d.jl` envuelve la función de acoplamiento:

```
struct OnInterface{F,T}
  f::F; nodes::Set{NTuple{2,Float64}}; zero::T; cache::Base.RefValue{Any}
end
function Gridap.Arrays.evaluate(s::OnInterface, x::Point)
  _key(x) in s.nodes || return s.zero        # fuera de la interfaz: valor descartado por Gridap
  s.cache[] === nothing && (s.cache[] = Gridap.Arrays.return_cache(s.f, x))
  evaluate!(s.cache[], s.f, x)               # caché de búsqueda reutilizada entre llamadas
end
interface_nodes = Set(_key(p) for c in get_cell_coordinates(Γair_int) for p in c)
uhair_int_(Λ) = OnInterface(interpolate_everywhere!(...), interface_nodes,
                            zero(VectorValue{2,Float64}))
```

Da los mismos valores que la versión anterior en los nodos de la interfaz: la solución final del run completo (20 pasos × 2 subpasos) con el backend de kernels coincide con la de Gridap a 1e-14, y ambas con el flujo de trabajo del script original. Es un parche: compara coordenadas redondeadas y vive solo en el ejemplo.

7. Arreglo propuesto en HyperFEM

Idea: el acoplamiento no se traduce a una función del espacio. Se define una prescripción `Coupled` que da a Gridap un marcador para la etiqueta y, tras el `TrialFESpace!` de Gridap, evalúa el campo solo en los nodos Dirichlet de la etiqueta y escribe los valores por máscara, conservando la caché de búsqueda de puntos entre llamadas. Es lo que hacía `InterpolableBC` antes del refactor,

integrado en el diseño nuevo de prescripciones. El código siguiente está probado como prototipo fuera de HyperFEM (`dev/proto_coupled.jl` de HyperFEMKernels, apartado 8); en HyperFEM faltaría solo encajarlo donde se indica.

BoundaryConditions.jl — nueva prescripción (junto a `Separable`, `NonSeparable` y `Parametric`):

```
"""
    Coupled(g)

Prescribed field `g(Λ)` (an `FEFunction`, `CellField` or `Interpolable`) that may be defined only on the
faces of its tag, e.g. a field of an `FESpace` built on an interface. Gridap receives a placeholder for
the tag; the field is evaluated solely at the Dirichlet nodes of the tag and written by mask after
Gridap's interpolation, keeping the point-search cache between calls. `g(Λ) === nothing` leaves the
values untouched (coupling not set yet).
"""
mutable struct Coupled{G} <: Prescription
    g::G
    mask::Union{Nothing,Vector{Bool}}     # Dirichlet dofs of the tag (set at first use)
    coords::Vector                        # coordinates of the nodes of those dofs
    ndofs::Int                            # dofs per node (components)
    cache::Any                            # Gridap point-search cache
    trian::Any                            # triangulation the cache was built for
end
Coupled(g) = Coupled(g, nothing, [], 0, nothing, nothing)
(p::Coupled)(Λ) = x -> x                  # placeholder for Gridap, overwritten by the mask (as the legacy code)
_derivative(p::Coupled, Λ, d) = d(p.g, Λ) # as Parametric: CellFields support subtraction

_field(f) = f
_field(f::Interpolable) = f.uh

"Mask and node coordinates of the Dirichlet dofs of `tag` in `space` (a TrialFESpace)."
function _coupling_nodes!(p::Coupled, space, tag::Int)
    mask = space.space.dirichlet_dof_tag .== tag
    U0 = TrialFESpace(space.space, [x -> x for _ in 1:num_dirichlet_tags(space.space)])  # values = coordinates
    dim = num_point_dims(get_triangulation(space))
    vals = get_dirichlet_dof_values(U0)[mask]
    p.ndofs = dim                                          # vector-valued fields: dim components per node
    p.coords = [VectorValue(vals[i:i+dim-1]...) for i in 1:dim:length(vals)]
    p.mask = mask
    p
end

"Write `p.g(Λ)` at the Dirichlet nodes of its tag into `space` (after Gridap's TrialFESpace!)."
function apply_coupling!(space, p::Coupled, tag::Int, Λ)
    f = p.g(Λ)
    f === nothing && return space
    p.mask === nothing && _coupling_nodes!(p, space, tag)
    fld = _field(f)
    trian = get_triangulation(fld)
    if p.cache === nothing || p.trian !== trian
        p.cache = return_cache(fld, p.coords[1]); p.trian = trian
    end
    dv = get_dirichlet_dof_values(space)
    idx = findall(p.mask)
    k = 0
    for x in p.coords
        v = evaluate!(p.cache, fld, x)
        for c in 1:p.ndofs
            dv[idx[k+c]] = v[c]
        end
        k += p.ndofs
    end
    space
end
```

BoundaryConditions.jl — los couplings heredados pasan a ser `Coupled`; `LegacyCoupling` deja de fabricar una función del espacio:

```
function _legacy_prescription(value, timestep)
    value === DirichletCoupling && return Coupled(LegacyCoupling(nothing))
    value isa InterpolableBC && return Coupled(LegacyCoupling(value.Interpolable))
    isnothing(timestep) && return Parametric(value)
    Separable(value, LegacyLoad(timestep))
end
```

```
(c::LegacyCoupling)(Λ) = isnothing(c.Interpolable) ? nothing : c.Interpolable(Λ)   # the field itself, not x -> ...

function InterpolableBC!(U::TrialFESpace, bc::BoundaryConditions, interface_tags::String, Interpolable)
    coupling = bc[findfirst(==(interface_tags), get_tags(bc))].value
    @assert coupling isa Coupled{LegacyCoupling} "Tag $interface_tags was not declared as a DirichletCoupling"
    coupling.g.Interpolable = Interpolable
    return coupling
end
# InterpolableBC (the struct) and _evaluator are no longer needed.
```

FESpaces.jl — aplicar los couplings después de la interpolación de Gridap (la variante incremental `TrialFESpace!(space, bc, Λ, ΔΛ)` y la multicampo llaman a esta y no cambian):

```
function Gridap.FESpaces.TrialFESpace!(space::SingleFieldFESpace, bc::BoundaryConditions, Λ::Real)
  TrialFESpace!(space, get_space_functions(bc, Λ))
  for (i, c) in enumerate(bc.conditions)
    c.value isa Coupled && apply_coupling!(space, c.value, i, Λ)
  end
  space
end

function Gridap.FESpaces.TrialFESpace(space::SingleFieldFESpace, bc::BoundaryConditions, Λ::Real=0.0)
  TrialFESpace!(TrialFESpace(space, get_space_functions(bc, Λ)), bc, Λ)
end
```

Con esto el camino nuevo recomendado también queda cubierto: en lugar de `Parametric(Λ -> Interpolable(uh))`, que tiene el mismo problema cuando uh vive solo en la interfaz, se escribe `BoundaryCondition("Interface", Coupled(Λ -> Interpolable(uh)))`.

Test — `BoundaryConditionsTests.jl`, el caso de uso real (campo definido solo sobre la interfaz):

```
# Coupling with a field that lives only on the interface (FESpace on a BoundaryTriangulation)
model = CartesianDiscreteModel((0.0, 1.0, 0.0, 1.0), (4, 4))
Γ = BoundaryTriangulation(model, tags="tag_8")                    # right edge
f(x) = VectorValue(x[2]^2, 0.1 * x[2])
uhΓ = interpolate_everywhere(f, FESpace(Γ, reffe2))              # undefined off Γ
coupled = BoundaryConditions(BoundaryCondition("tag_7", [0.0, 0.0]),
                             BoundaryCondition("tag_8", Coupled(Λ -> Interpolable(uhΓ))))
Vc = TestFESpace(model, reffe2, coupled)
Uc = TrialFESpace(Vc, coupled, 1.0)                               # must not evaluate uhΓ off Γ
exact = get_dirichlet_dof_values(TrialFESpace(Vc, (VectorValue(0.0, 0.0), f)))
@test get_dirichlet_dof_values(Uc) ≈ exact
get_free_dof_values(uhΓ) .*= 2.0                                  # field updated in place (staggered loop)
TrialFESpace!(Uc, coupled, 1.0)
@test get_dirichlet_dof_values(Uc) ≈ get_dirichlet_dof_values(TrialFESpace(Vc, (VectorValue(0.0, 0.0), x -> 2f(x))))
# legacy path
legacy = DirichletBC(["tag_7", "tag_8"], [[0.0, 0.0], DirichletCoupling], [Λ -> 1.0, nothing])
Ul = TrialFESpace(TestFESpace(model, reffe2, legacy), legacy, 0.0)
InterpolableBC!(Ul, legacy, "tag_8", Λ -> uhΓ); TrialFESpace!(Ul, legacy, 1.0)
@test get_dirichlet_dof_values(Ul) ≈ get_dirichlet_dof_values(Uc)
```

8. Validación del prototipo

`dev/proto_coupled.jl` (HyperFEMKernels, PR #7) contiene exactamente el código de `Coupled`, `_coupling_nodes!` y `apply_coupling!` del apartado 7 y lo aplica al espacio del vacío del ejemplo (`Short_magnet.msh`, etiquetas Fixed_Air, Fixed_Air_x, Interface), con un campo analítico interpolado en un FESpace construido solo sobre la interfaz, frente a la interpolación directa de Gridap de la misma función analítica en la etiqueta:

Magnitud	**Valor**
dofs libres / Dirichlet del espacio del vacío	15 580 / 852
nodos de la interfaz	205
diferencia relativa frente a Gridap (valores Dirichlet)	5,9e-19
primera aplicación (construye máscara, coordenadas y caché)	0,95 s
aplicaciones siguientes	0,0002 s (1 MB)
mismo coste con la implementación actual (evaluación sin caché)	0,78 s (1 366 MB)
campo modificado in situ (×2): sigue el cambio sin reconstruir nada; etiquetas restantes intactas	sí

Límites del prototipo: supone campos vectoriales con `dim` componentes por nodo (como el código antiguo); para un campo escalar acoplado habría que leer el número de componentes del `reffe`. Las máscaras parciales (`dirichlet_masks` con componentes libres en la etiqueta acoplada) tampoco se contemplan, igual que antes. Scripts afectados en Adv_Mater_Ultra_Soft_Magnet: `hMRE_Vacuum_2D.jl` y `hMRE_Vacuum&magnet_2D.jl`.

Referencias

HyperFEM.jl: commits 3e005d3 (refactor de prescripciones), e161faa (soporte de couplings heredados), fa414c6 (último estado anterior); ficheros `src/ComputationalModels/BoundaryConditions.jl` y `FESpaces.jl`. HyperFEMKernels.jl: PR #7 (`examples/hmre_vacuum_2d`, `dev/prof_hmre_vacuum_bc.jl`, `dev/prof_hmre_vacuum_solve.jl`), entrada del 21-10-2026 en ESTADO.md. Cifras: medida nuestra (Mac M5 Max, 18 hilos, Julia 1.13, Gridap 0.19.9).